Atividade Prática - Aula 12

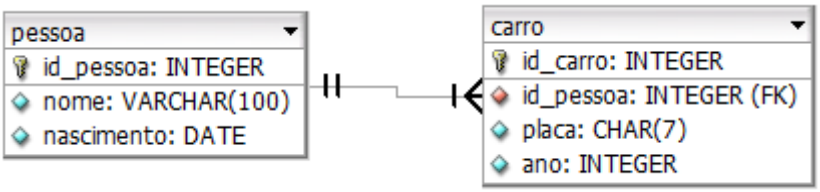
Tema - Índices

Entrega da Atividade

1. **Produção**:
   a) Crie um script **schema.sql** com todos os comandos SQL.
   b) Print da tela inteira, **incluindo a barra de tarefas do Windows**.
2. Exporte/salve o arquivo de acordo com o padrão: **BDR-Atividade-Aula12.pdf**
3. Enviar a Atividade em PDF no GitHub:
   a) Acesse seu repositório da disciplina: **FATEC-JCR-2DSM-BDR-2026-1-seunome**
   b) Se ainda não existir, você deverá criar a pasta da atividade: **P2-Conteudos/Atividades/Atividade-Aula12/README.md**
   c) Faça o **commit** do pdf da atividade feita na pasta.
   d) Em seguida copie o **link** do pdf.
   e) Acesse o **Kanban-BDR-2026-1** da disciplina que se encontra no repositório da professora.
   f) Clique no card: **BDR-Atividade-Aula12**.
   g) No comentário, cole o **link** do **pdf** da atividade que você fez.
   h) Volte ao seu repositório e acesse o seu **Kanban-BDR-2026-1.**
   i) Mova o card **BDR-Atividade-Aula12** para a coluna **Entregue**.

Enunciados da Atividade

Para a correta execução dos exercícios, utilize os comandos a seguir para criar as tabelas e carregar os registros a partir de arquivos CSV.



Cada arquivo contém 100 mil registros.

Antes de executar os comandos COPY, é necessário ajustar o caminho dos arquivos de acordo com a localização dos CSVs no seu computador.

```
DROP TABLE IF EXISTS carro, pessoa;
CREATE TABLE IF NOT EXISTS pessoa (
    id_pessoa INTEGER PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    nascimento DATE
);

CREATE TABLE IF NOT EXISTS carro (
   id_carro INTEGER PRIMARY KEY,
   placa CHAR(7) NOT NULL,
   ano INTEGER,
   id_pessoa INTEGER NOT NULL,
   FOREIGN KEY (id_pessoa)
      REFERENCES pessoa(id_pessoa)
      ON DELETE CASCADE
);
```

```
COPY pessoa (id_pessoa, nome,
nascimento)
FROM 'C:/caminho/aula3_pessoa.csv'
DELIMITER ','
CSV HEADER;

COPY carro (id_carro, placa, ano,
id_pessoa)
FROM 'C:/caminho/aula3_carro.csv'
DELIMITER ','
CSV HEADER;
```

Comando para listar os índices de uma tabela:

```
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'pessoa';
```

Exercício 1 – Observando o impacto de um índice (B-tree) com EXPLAIN ANALYZE

Objetivo: Compreender, por meio de medições, a diferença entre varredura sequencial (*Seq Scan*) e acesso por índice (*Index Scan*), utilizando a coluna `nome` da tabela `pessoa`.

Parte A – Consulta sem índice explícito

1. Execute a consulta abaixo e registre o tipo de varredura e o tempo retornado:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Silva';
```

2. Execute uma segunda vez com outro nome comum, por exemplo:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'João Santos';
```

Parte B – Criação do índice

3. Crie um índice B-tree na coluna nome:

```
CREATE INDEX idx_pessoa_nome
ON pessoa (nome);
```

Parte C – Consulta com índice

4. Execute novamente as duas consultas da Parte A.

Parte D – Responda, de forma objetiva:

- Qual plano foi utilizado antes do índice?
- Qual plano foi utilizado após o índice?
- Houve redução no tempo de execução?
- O índice foi utilizado em ambos os nomes testados? Justifique com base na saída do plano.

Exercício 2 – Índice, Seletividade e Decisão do Otimizador

Objetivo: Analisar como a seletividade de uma coluna influencia o uso (ou não) de um índice pelo PostgreSQL, mesmo quando o índice existe.

Enunciado:

No Exercício 1, foi criado um índice B-tree na coluna nome da tabela pessoa, e observou-se uma redução significativa no tempo de execução das consultas.

Neste exercício, o objetivo é verificar se o índice **continua sendo vantajoso** quando a consulta retorna **uma grande quantidade de registros**, isto é, quando a seletividade da condição é baixa.

Parte A – Consulta por intervalo sem índice explícito

1. Remova o índice criado no exercício anterior (caso ainda exista):

```
DROP INDEX IF EXISTS idx_pessoa_nome;
```

2. Execute a consulta abaixo e observe o plano de execução:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento >= DATE '1970-01-01';
```

Parte B – Criação de índice na coluna nascimento

3. Crie um índice B-tree na coluna nascimento:

```
CREATE INDEX idx_pessoa_nascimento
```

```
ON pessoa (nascimento);
```

Parte C – Consulta com índice criado

4. Execute novamente a consulta da Parte A.

Parte D – Responda, de forma objetiva

- O índice criado foi utilizado pelo PostgreSQL?
- O plano de execução mudou após a criação do índice?
- Houve melhora significativa no tempo de execução?
- Por que o otimizador pode optar por **não utilizar um índice**, mesmo quando ele existe?

Exercício 3 – Índice Composto e Ordem das Colunas

Objetivo: Compreender o funcionamento de **índices compostos**, analisando:

- quando eles são utilizados;
- a importância da **ordem das colunas** no índice;
- a diferença entre índice simples e índice composto na tomada de decisão do otimizador.

Enunciado:

Nos exercícios anteriores, foram analisados índices simples criados sobre uma única coluna.

Neste exercício, o objetivo é analisar o comportamento do PostgreSQL ao utilizar um **índice composto**, criado sobre as colunas nascimento e nome da tabela pessoa.

Parte A – Consulta sem índice composto

1. Remova o índice criado no exercício anterior (caso ainda exista):

```
DROP INDEX IF EXISTS idx_pessoa_nascimento;
```

2. Execute a consulta abaixo e observe o plano de execução:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento >= DATE '2000-01-01'
  AND nome = 'Ana Silva';
```

Parte B – Criação do índice composto

3. Crie um índice composto nas colunas nascimento e nome, nesta ordem:

```
CREATE INDEX idx_pessoa_nascimento_nome
ON pessoa (nascimento, nome);
```

Parte C – Consulta com índice composto

4. Execute novamente a consulta da Parte A.

Parte D – Teste da ordem das colunas

5. Execute a consulta abaixo:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nome = 'Ana Silva';
```

Parte E – Responda, de forma objetiva

- Qual plano foi utilizado antes da criação do índice composto?
- Qual plano foi utilizado após a criação do índice composto?
- O índice composto foi utilizado na consulta que filtra **apenas por nome**?
- Por que a **ordem das colunas** no índice composto é relevante?

Exercício 4 – Uso de Múltiplos Índices Simples (BitmapAnd)

Objetivo: Compreender como o PostgreSQL pode combinar **múltiplos índices simples** para atender a uma consulta com mais de uma condição, utilizando o operador interno **BitmapAnd**, e comparar essa estratégia com o uso de um índice composto.

Enunciado:

No Exercício 3, foi analisado o uso de um **índice composto** para otimizar consultas com múltiplas condições.

Neste exercício, o objetivo é analisar um cenário alternativo, no qual **índices simples separados** são criados para cada coluna, e o PostgreSQL decide combiná-los durante a execução da consulta.

Parte A – Preparação do ambiente

1. Remova o índice composto criado no exercício anterior (caso ainda exista):

```
DROP INDEX IF EXISTS idx_pessoa_nascimento_nome;
```

2. Crie dois **índices simples**, um para cada coluna:

```
CREATE INDEX idx_pessoa_nascimento
ON pessoa (nascimento);
CREATE INDEX idx_pessoa_nome
ON pessoa (nome);
```

Parte B – Consulta utilizando múltiplas condições

3. Execute a consulta abaixo e observe atentamente o plano de execução:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento >= DATE '2000-01-01'
  AND nome = 'Ana Silva';
```

Parte C – Responda, de forma objetiva

- O PostgreSQL utilizou os dois índices simples na consulta?
- Qual é o papel do operador BitmapAnd no plano de execução?
- O que o plano efetivamente fez?

Exercício 5 – Índice para Consulta com Filtro por Intervalo

Considere a consulta abaixo, executada sobre a tabela carro:

```
EXPLAIN ANALYZE
SELECT *
FROM carro
WHERE ano BETWEEN 2015 AND 2020;
```

Tarefa:

Atenção: Antes de prosseguir, exclua os índices criados nos exercícios anteriores.

1. Crie o índice mais adequado para esse caso.
2. Execute novamente a consulta com EXPLAIN ANALYZE e compare o plano obtido antes e após a criação do índice.

Exercício 6 – Índice para Otimização de JOIN

Objetivo: Compreender a importância de índices em chaves estrangeiras para melhorar o desempenho de operações de junção (JOIN).

Enunciado:

Analise a consulta a seguir, que lista o nome da pessoa e a placa de seus carros:

```
EXPLAIN ANALYZE
SELECT p.nome, c.placa
```

```
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE p.nome = 'Ana Silva';
```

Tarefa:

Atenção: Antes de prosseguir, exclua os índices criados nos exercícios anteriores.

1. Crie os índices necessários para melhorar o desempenho da consulta.
2. Execute novamente a consulta e analise as alterações no plano de execução.

Exercício 7 – Índice Composto em Consulta com JOIN e Filtro

Objetivo: Projetar um índice composto para otimizar simultaneamente um JOIN e um filtro por intervalo.

Enunciado:

Considere a consulta abaixo:

```
EXPLAIN ANALYZE
SELECT p.nome, c.placa, c.ano
FROM pessoa p
JOIN carro c ON p.id_pessoa = c.id_pessoa
WHERE p.nascimento >= DATE '1980-01-01'
  AND c.ano >= 2018;
```

Essa consulta retorna apenas uma pequena parcela dos registros, mas apresenta custo elevado sem índices adequados.

Tarefa:

Atenção: Antes de prosseguir, exclua os índices criados nos exercícios anteriores.

1. Analise quais colunas participam:
   - do JOIN;
   - dos filtros da cláusula WHERE.

   Proponha e crie os índices necessários para melhorar o desempenho da consulta.

   Justifique a escolha entre índices simples ou compostos.

Exercício 8 – Uso de Índice GiST

Até o momento, foram utilizados índices do tipo B-tree, que são adequados para a maioria das consultas por igualdade e intervalo.

Neste exercício, o objetivo é explorar o uso de um índice do tipo GiST aplicado à coluna nascimento da tabela pessoa.

Atenção: Antes de prosseguir, exclua os índices criados nos exercícios anteriores.

Parte A – Consulta sem índice GiST

1. Execute a consulta abaixo e observe o plano de execução:

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';
```

Parte B – Criação do índice GiST

2. Caso ainda não esteja habilitada, ative a extensão necessária para suporte a GiST em tipos escalares:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
```

3. Crie um índice do tipo **GiST** sobre a coluna nascimento:

```
CREATE INDEX idx_pessoa_nascimento_gist
ON pessoa
USING GIST (nascimento);
```

4. Certifique que o um índice foi criado:

```
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'pessoa';
```

Parte C – Consulta com índice GiST

5. Execute novamente a consulta da Parte A.

Parte D – Responda, de forma objetiva

- Qual plano foi utilizado antes da criação do índice GiST?
- O índice GiST foi utilizado após a criação?
- Houve melhora no tempo de execução?